Mini Transformer Benchmark Report

Yi Yi Htun - 68070701604 (https://github.com/YiYiHtun/mini_transformer_benchmark)

1. Introduction

This assignment aims to implement a mini Transformer model from scratch and evaluate its performance on a synthetic sequence classification task.The goal is to understand key components of the Transformer architecture, including attention mechanisms and positional encoding, and analyze their impact on model performance.

2. Method

2.1 Problem Definition

Given a sequence of tokens, the task is **to predict whether the first non-padding token appears again in the second half of the sequence.**

2.2 Model Architecture

The model consists of:

- Token Embedding
- Positional Encoding
- Transformer Encoder Blocks
- Mean Pooling over valid tokens
- Linear Classification Head

2.3 Attention Mechanism

Scaled dot-product attention is implemented as:

- Query, Key, Value projections
- Attention score computation
- Softmax normalization
- Weighted sum of values

Multi-head attention splits the embedding into multiple heads to capture different relationships.

3. Experimental Setup

Experiments were run for 10 epochs using the same dataset split and training pipeline. The dataset contains sequences with up to 20 tokens and corresponding padding masks. All

Transformer variants used a model dimension of 64, a feed-forward dimension of 128, a dropout rate of 0.1, and the Adam optimizer with learning rate 1e-3. The batch size was 32.

```python
if __name__ == "__main__":
    parser = argparse.ArgumentParser()

    parser.add_argument("--train_path", type=str, default="train.csv")
    parser.add_argument("--val_path", type=str, default="validation.csv")
    parser.add_argument("--test_path", type=str, default="test.csv")
    parser.add_argument("--output_dir", type=str, default="outputs")
    parser.add_argument("--run_name", type=str, default="run1")

    parser.add_argument("--model_type", type=str, choices=["simple", "transformer"], default="simple")
    parser.add_argument("--seed", type=int, default=42)
    parser.add_argument("--batch_size", type=int, default=32)
    parser.add_argument("--lr", type=float, default=1e-3)
    parser.add_argument("--epochs", type=int, default=10)

    parser.add_argument("--d_model", type=int, default=64)
    parser.add_argument("--ff_dim", type=int, default=128)
    parser.add_argument("--num_heads", type=int, default=4)
    parser.add_argument("--num_layers", type=int, default=1)
    parser.add_argument("--dropout", type=float, default=0.1)

    parser.add_argument(
        "--use_positional_encoding",
        action="store_true",
        help="Include this flag to enable positional encoding."
    )
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                    powershell

Parameters: 385
(.venv) PS D:\KMUTT\KMUTT_Master_syllabus\CPE NLP\mini_transformer_benchmark> python train.py --model_type transformer --run_name transformer_1 --epo
chs 10 --d_model 64 --ff_dim 128 --num_heads 4 --num_layers 1 --dropout 0.1 --use_positional_encoding
Epoch 01 | Train Loss: 0.4459 | Train Acc: 0.8198 | Val Loss: 0.4584 | Val Acc: 0.8140
Epoch 02 | Train Loss: 0.4125 | Train Acc: 0.8210 | Val Loss: 0.3858 | Val Acc: 0.8270
Epoch 03 | Train Loss: 0.3844 | Train Acc: 0.8398 | Val Loss: 0.3597 | Val Acc: 0.8350
Epoch 04 | Train Loss: 0.3290 | Train Acc: 0.8652 | Val Loss: 0.2625 | Val Acc: 0.9130
Epoch 05 | Train Loss: 0.2659 | Train Acc: 0.8960 | Val Loss: 0.2473 | Val Acc: 0.8970
Epoch 06 | Train Loss: 0.1982 | Train Acc: 0.9260 | Val Loss: 0.1685 | Val Acc: 0.9340
Epoch 07 | Train Loss: 0.1506 | Train Acc: 0.9456 | Val Loss: 0.2468 | Val Acc: 0.9160
Epoch 08 | Train Loss: 0.1269 | Train Acc: 0.9516 | Val Loss: 0.1243 | Val Acc: 0.9540
Epoch 09 | Train Loss: 0.1066 | Train Acc: 0.9616 | Val Loss: 0.1165 | Val Acc: 0.9590
Epoch 10 | Train Loss: 0.0910 | Train Acc: 0.9672 | Val Loss: 0.1078 | Val Acc: 0.9700

Finished.
Saved model to: outputs\transformer_1.pt
Saved metrics to: outputs\transformer_1_metrics.json
Test Acc: 0.9700
Training Time: 25.15 sec
Parameters: 33857
```

The evaluated variants are:

- A: Simple baseline model without positional encoding.
- B: Transformer with positional encoding, 1 head, 1 layer.
- C: Transformer with positional encoding, 4 heads, 1 layer.
- D: Transformer without positional encoding, 4 heads, 1 layer.
- E: Transformer with positional encoding, 4 heads, 2 layers.

4. Results

Model	Positional Encoding	Heads	Layers	Val Acc	Test Acc	Train Time
A	No	-	-	0.814	0.823	0.07 min
B	Yes	1	1	0.901	0.895	0.41 min
C	Yes	4	1	0.970	0.970	0.45 min

D	No	4	1	0.814	0.823	0.37 min
E	Yes	4	2	0.987	0.973	0.67 min

The following table summarizes validation accuracy, test accuracy, and training time for each model variant. Training time is reported from the saved metrics in the outputs directory.

5. Discussion

The results show that Transformer models perform much better than the simple baseline model on this task.

- The baseline (Model A) only achieved about 0.81 validation accuracy, while the best Transformer model (Model E) reached 0.98 validation accuracy.
- Positional encoding plays a very important role.
  Model C (with positional encoding) achieved 0.97 accuracy, while Model D (without positional encoding) stayed at 0.81 and did not improve.
  This shows that the model needs position information to understand the sequence.
- Increasing the number of layers also improves performance.
  Model E (2 layers) performed slightly better than Model C (1 layer), but it also took more time to train. This shows a trade-off between accuracy and training cost.
- Using more attention heads also helps.
  Model B (1 head) achieved 0.90 accuracy, while Model C (4 heads) reached 0.97.
  This means multiple heads allow the model to learn better relationships in the sequence.

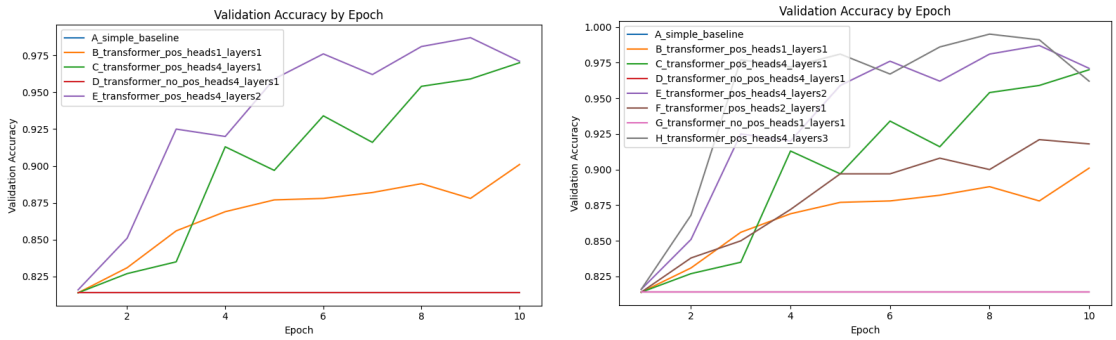

Figure 1: Validation accuracy curves by epoch for each experiment.

From the accuracy curves, Transformer models learn faster and more smoothly than the baseline. The baseline stops improving early, while Transformer models continue to improve over time.

6. Conclusion

This assignment demonstrated how Transformer models work internally.
We observed that positional encoding and multi-head attention significantly improve performance. The experiment provided practical understanding of attention mechanisms and sequence modeling.